

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Progetto d'esame- Corso di Programmazione e Strutture Dati

***Sistema di gestione delle
segnalazioni in un comune***

Salerno Mattia
NF12100434
A.A 2026/2027

INDICE

1	Presentazione del progetto.....	pag.3
2	Scelta dell'ADT.....	pag.4
	2.1 Ottimizzazione dell'inserimento.....	
	2.2 Ottimizzazione delle altre funzioni.....	
3	Progettazione.	pag.7
	3.1 Avvio.....	
	3.2 Inserimento.....	
	3.3 Stampa delle più urgenti.....	
	3.4 Ricerca filtrata.....	
	3.5 Ricerca tramite id.....	
	3.6 Rimovibilità.....	
	3.7 Modifica dello stato.....	
	3.8 Eliminazione.....	
	3.9 Generazione del report.....	
4	Specifica sintattica e semantica.....	pag. 11
	4.1 ADT Data.....	
	4.2 ADT Segnalazione.....	
	4.3 ADT PQueue.....	
	4.4 ADT hashtable.....	
	4.5 File Gestione_Main.....	
5	Razionale dei Casi di Test.....	pag.21
	5.1 Dati errati e corretto inserimento.....	
	5.2 Ricerca per id.....	
	5.3 Cambiamento di stato.....	
	5.4 Priorità.....	
	5.5 Ricerca filtrata.....	
	5.6 Report.....	
	5.7 Eliminazione.....	
	5.8 Visione delle più urgenti.....	

1 Presentazione del progetto

Questo progetto, realizzato in Windows, consiste in un sistema di gestione delle segnalazioni all'interno di un comune. Esso sarà in grado di registrarle, eliminarle quando permesso, modificarne lo stato e visualizzarle secondo determinati filtri. Il programma è destinato non ai cittadini ma agli operatori del comune, che riceveranno tramite sistema separato le segnalazioni e che si occuperanno della loro gestione.

Le segnalazioni avranno diversi dati a loro associati, ovvero:

- Codice identificativo
- Nome e Cognome del cittadino segnalante
- Categoria
- Descrizione
- Data
- Urgenza
- Stato
- Flag per la rimovibilità

Ognuna di esse sarà inseribile nel sistema sia tramite input da tastiera che tramite file di testo, e sarà visualizzabile tramite id; anche il cambiamento di stato, del flag e l'eliminazione richiederanno il codice identificativo. Sarà inoltre possibile visualizzare tutte le segnalazioni, così come solo alcune in base al filtro che può essere per id, stato, urgenza e categoria, inoltre l'utente può richiedere di visionare un numero definito di segnalazioni tra quelle a più alta priorità. Infine sarà anche presente la possibilità di generare un report, contenente il numero di: segnalazioni presenti, segnalazioni per stato, per urgenza e per categoria, mostrando poi quella più presente.

2 Scelta del Tipo di Dato Astratto (ADT)

La scelta dell'ADT nasce da un'analisi del contesto nel quale il programma verrà usato.

L'inserimento di nuove segnalazioni così come l'eliminazione è un'operazione la cui frequenza varia da comune a comune, a differenza della ricerca di singole segnalazioni o delle ricerche per filtri che saranno presumibilmente le scelte principali. In particolare le funzioni di modifica e di ricerca richiedono a loro volta la ricerca tramite id, che sarà quindi la funzione alla quale dare maggiore priorità.

Allo stesso modo, la visualizzazione di tutte le segnalazioni sarà un'azione scelta poco frequentemente, come la generazione del report.

2.1 Ottimizzazione dell'inserimento

Tra i vari tipi di dati astratti, è importante analizzare la loro complessità di computazione al compiere di varie azioni.

Per quanto riguarda l'inserimento dei dati, spicca sicuramente la tabella Hash, con complessità computazionale di ricerca $O(1)$.

Complessità di inserimento delle varie strutture.

Strutture dati	Lista ordinata	BST	Coda con priorità	Hash
Complessità di inserimento	$O(n)$	$O(\log n)$, $O(n)$ nel caso peggiore	$O(\log n)$	$O(1)$

Nonostante ciò, la scelta per la struttura principale non ricade sulla tabella hash, questo perché non ha la capacità di mantenere un ordine tra i suoi elementi; questa funzionalità è necessaria nel programma sia per la stampa delle segnalazioni più urgenti, sia per la stampa di tutte le segnalazioni che seppur non necessario avverrà in ordine di urgenza, questo per decisione stilistica ma anche realistica. La scelta si divide quindi tra coda con

priorità e BST, e la migliore scelta tra le due è la prima: infatti in uno scenario realistico la struttura BST rimarrebbe facilmente sbilanciato a causa della maggiore frequenza di segnalazioni con urgenze basse (semplici piccole segnalazioni giornaliere) e della natura prioritaria della data, che sarà quasi sempre più recente delle precedenti (a priorità di urgenza vige il FIFO, first in first out). Proprio per quest'ultimo motivo, la complessità reale di inserimento nella coda prioritaria sarà spesso minore di $O(\log n)$.

2.2 Ottimizzazione delle altre funzioni

Andando però ad analizzare la complessità delle altre azioni, è facile notare come le code con priorità abbiano evidenti punti deboli.

Complessità delle varie azioni, dove k è il numero di segnalazioni da visualizzare.

	Coda con priorità	BST	Hash
Inserimento	$O(\log n)$	$O(\log n)$, $O(n)$ nel caso peggiore	$O(1)$
Eliminazione	$O(n)$ per trovare la segnalazione + $O(\log n)$	$O(\log n)$, $O(n)$ nel caso peggiore	$O(1)$
Stampa completa	$O(n)$ (senza tenere ordine)	$O(n)$	$O(n)$ (senza tenere ordine)
Ricerca per id	$O(n)$	$O(n)$	$O(n)$
Ricerca per categoria/stato/urgenza	$O(n)$	$O(n)$	$O(n)$
Stampa delle più urgenti	$O(1)$ se solo la prima, altrimenti $O(n)+O(k \log n)$	$O(k)$	$O(n)$

L'hash risulta la scelta perfetta da implementare insieme alla coda con priorità per quanto riguarda la ricerca tramite id, la funzione più importante di tutto il programma.

Al contempo per ovviare alla lentezza della ricerca per categoria, urgenza o stato è possibile implementare 3 array di liste concatenate, dove ogni lista contiene l'id delle segnalazioni con il criterio selezionato. Esse verranno aggiornate all'inserimento e alla rimozione di ogni segnalazione, così come alla modifica dello stato (essendo l'unica categoria da poter modificare). In questa maniera il tempo di computazione passerà da $O(n)$ a $O(k)$, con k pari al numero delle segnalazioni con quel criterio.

3 Progettazione

Prima di esaminare le funzionalità, è necessaria una visione generale sulle ADT usate.

Queste sono **Data**, **Segnalazione**, **hashtable** e **PQueue**:

- **Data** è un puntatore alla struttura **data**, nascosta e contenente giorno mese e anno, garantendone la validità.
- **Segnalazione** è un puntatore alla struttura **segnalazione**, anch'essa nascosta e contenente una serie di dati, ovvero codice id, nome, cognome, categoria, descrizione, data, urgenza, stato e flag per la rimovibilità. Rappresenta ovviamente una singola segnalazione, permette di gestirla con sicurezza, e viene inizializzata con il flag posto a 0 (non rimovibile).
- **hashtable** è un puntatore alla struttura **hash**, ancora una volta nascosta e contenente la dimensione della tabella e un array di puntatori a strutture item. Ognuno di questi memorizza al suo interno la chiave di ricerca, ovvero l'ID della segnalazione, la sua posizione corrente all'interno della coda con priorità, e un puntatore al nodo successivo per la gestione delle collisioni tramite concatenamento.
- **PQueue** è anch'esso un puntatore a una struttura, in questo caso **c_PQ**. Di nuovo, la struttura è nascosta e contiene un array di strutture Segnalazione, il numero di elementi, la tabella Hash a esso associata e 3 array di liste con dimensione pari ognuna al numero delle possibili scelte del dato che rappresentano (ovvero urgenza stato e categoria) + 1, per semplificare l'inserimento.

Tra questi l'ADT principale è la PQueue, che si occuperà di contenere tutte le segnalazioni in maniera completa; intanto l'hashtable fornirà supporto per le ricerche tramite id, rendendole istantanee e gli array delle liste diminuiranno la complessità della ricerca filtrata.

3.1 Avvio

All'avvio del programma viene stampato sul terminale un menù nel quale l'utente, tramite una selezione numerica fornita in input da tastiera, sceglie l'azione che desidera svolgere.

3.2 Inserimento

Per quanto riguarda l'inserimento delle segnalazioni, sono presenti due scelte:

- La prima scelta consiste nell'inserimento tramite file di testo, l'utente ne fornirà il nome (es. segnalazioni.txt) qualora esso sia all'interno della cartella del progetto, o alternativamente la directory.
- La seconda scelta è l'inserimento manuale da tastiera dei singoli dati di una segnalazione. Il codice id è manualmente inserito, idealmente per supportare un eventuale trasferimento di segnalazioni già create in forma cartacea; selezionando "0" esso verrà creato in maniera automatica, per poi essere comunicato prima della creazione effettiva.

In entrambe le scelte ogni dato inserito sarà soggetto al controllo della sua validità.

Sempre all'inserimento, una volta terminato l'id verrà inserito anche nei 3 array di liste, più specificamente nelle liste delle segnalazioni che hanno lo stesso stato, categoria o urgenza. Questo passaggio usa relativamente poca memoria in più, la complessità di computazione rimane quasi invariata ($+O(1)$ per ogni filtro, tenendole non ordinate), ma ha il potenziale per ridurre in maniera drastica la complessità della ricerca che diventa $O(k)$ con k che è il numero di elementi con il determinato filtro.

3.3 Stampa delle più urgenti

La stampa delle funzioni più urgenti è una funzione fondamentale, in particolare la visione della più urgente, e la coda eccelle in ciò con una complessità di $O(1)$. Alternativamente, qualora sia richiesta la stampa di

più di una funzione, possono essere applicati due approcci diversi: uno di questi è copiare l'intera struttura, stampare il primo elemento ed eliminarlo, questo per il numero di elementi che si desidera visionare. Il costo totale dell'azione si traduce quindi in $O(n) + O(k \log n)$ dove k sono gli elementi da stampare.

L'altro approccio è creare un array di segnalazioni temporaneo, stampare il primo elemento, posizionarlo nell'array ed eliminarlo dalla coda, questo sempre per k volte. Finite le stampe, gli elementi dell'array vengono inseriti uno ad uno nella coda; il costo dell'azione è quindi $O(k) + O(k \log n) + O(k \log n)$. Andando a paragonare le due funzioni, si può trovare la relazione tra k ed n tale che sia più conveniente usare un metodo o l'altro: risulta infatti $k/n < 1/(1 + \log n)$, disequazione che quando rispettata indica che il secondo metodo (l'uso dell'array temporaneo) è maggiormente efficace. La differenza tra i due metodi non è significativa con un numero poco elevato di segnalazioni, ma qualora esso cresca, la complessità risparmiata per visionare un numero molto basso di segnalazioni cresce esponenzialmente. Per rendere chiaro il concetto, grazie alla differenza tra questi due metodi se un utente volesse visionare 20 segnalazioni quando nel database ne sono presenti 1000, il metodo della copia completa ha complessità di circa 1200 mentre il secondo solo di circa 420.

3.4 Ricerca filtrata

La ricerca filtrata come citato nel capitolo 3.2, avviene tramite la semplice stampa della lista contenente le segnalazioni con il filtro desiderato, inserito ancora una volta in input dall'utente. È presente un sistema di controllo per la validità dei dati inseriti.

3.5 Ricerca tramite id

La ricerca per id, funzione necessaria anche per le modifiche alle segnalazioni, avviene sfruttando la tabella hash. La funzione dopo aver ricevuto il codice id, trova istantaneamente l'item corrispondente nella tabella. Questo restituisce la posizione nella coda alla funzione di ricerca, che a sua volta restituisce alla funzione di stampa o di modifica la segnalazione cercata. Qualora il codice non abbia corrispondenza o sia

errato, verrà mostrato a schermo l'avviso. Ogni spostamento nella coda, che sia a causa di salita, discesa, inserimenti ed eliminazioni, necessita quindi dell'aggiornamento nella tabella della posizione attuale.

3.6 Rimovibilità

La modifica del flag sfrutta la ricerca per id, andando semplicemente a modificare da 0 a 1 il valore, e se questo è già 1 viene avvisato l'utente ed esso non cambia. Il motivo dell'implementazione di questo dato, e quindi della funzione, è garantire che una segnalazione sia rimovibile solo se consentito, fornendo inoltre uno strumento di prevenzione per eliminazioni accidentali. È presente un sistema di controllo per la validità dei dati inseriti.

3.7 Modifica dello stato

La modifica dello stato ricevendo l'id della segnalazione da modificare e lo stato da cambiare, userà la ricerca per id e tenterà di applicare la modifica. Qualora il valore sia già quello desiderato, verrà mostrato il messaggio a schermo. Nel momento in cui lo stato viene modificato a 3, ovvero chiusa, l'utente potrà scegliere se eliminare già la segnalazione in questione o se lasciarla ancora all'interno del sistema, questo in prospettiva della necessità dell'utente di avere la possibilità di riaprire la segnalazione, modificarne dei dati o più semplicemente far sì che essa sia visionabile, magari insieme ad altre segnalazioni già chiuse. Quando lo stato viene modificato, viene aggiornata anche la sua posizione all'interno dell'array di liste dedicato. È presente un sistema di controllo per la validità dei dati inseriti.

3.8 Eliminazione

L'eliminazione è un'operazione che, ricevuto l'id, necessita della variabile flag corrispondente a 1. All'eliminazione, il dato andrà rimosso dagli array di liste per i filtri, dalla tabella hash e dalla coda, che si dovrà quindi riordinare. È presente un sistema di controllo per la validità dei dati inseriti.

3.9 Generazione del report

La stampa del report rappresenta una delle azioni meno frequenti ma più utili per lo scopo del programma, è infatti fondamentale per un comune tenere conto

di tutte le segnalazioni per capire dove sono le criticità principali e come stanno venendo gestite, cioè se la maggior parte di esse sono in lavorazione o meno.

4 Specifica sintattica e semantica

Segue ora la specifica sintattica e semantica dei vari ADT e dei loro operatori.

4.1 ADT Data

Specifica sintattica

Tipi di riferimento: Data

Specifica semantica

Tipo di riferimento: il tipo Data è l'insieme di 3 valori int rappresentanti giorno, mese e anno.

4.2 ADT Segnalazione

Specifica sintattica

Tipi di riferimento: Segnalazione

Tipi usati: int, char*

Operatori : newSegnalazione (int, char*, char*, int, char*, int, int, int, int, int, int) → Segnalazione

Specifica semantica

Tipo di riferimento: Segnalazione è il tipo composto da 9 elementi, ovvero id, nome, cognome, categoria, descrizione, Data , urgenza, stato e flag di rimovibilità.

Questi sono rispettivamente int, char*, char*, int, char*, Data, int, int, int.

Operatori:

newSegnalazione (id, nome, cognome, cat, desc, giorno, mese, anno, urg, stato, rimuovibile)->s

-Pre: $id > 0$, $1 \leq cat \leq 5$, $1 \leq urg \leq 5$, $1 \leq stato \leq 3$, (giorno, mese, anno) validi, rimuovibile=0 o 1.

-Post: s=(id, nome, cognome, cat, desc, Data , urg, stato, rimuovibile)

4.3 ADT PQueue

Specifica sintattica

Tipi di riferimento: PQueue

Tipi usati: Segnalazione, int, char*, void

Operatori: newPQ () $\rightarrow$ PQueue

emptyPQ (PQueue) $\rightarrow$ int

DestroyPQueue (PQueue) $\rightarrow$ void

insert (PQueue, Segnalazione) $\rightarrow$ int

delete (PQueue, int) $\rightarrow$ int

visualizzaTutte (PQueue) $\rightarrow$ void

visualizzaPerStato (PQueue, int) $\rightarrow$ void

visualizzaPerUrgenza (PQueue, int) $\rightarrow$ void

visualizzaPiuUrgenti (PQueue, int) $\rightarrow$ void

visualizzaPerCodice (PQueue, int) $\rightarrow$ void

visualizzaPerCategoria (PQueue, int) $\rightarrow$ void

rendi_rimovibile (PQueue, int) $\rightarrow$ int

cambia_stato (PQueue, int, int) $\rightarrow$ int

caricaDaFile (PQueue, char*) $\rightarrow$ int

insertSegnalazione (PQueue, int, char*, char*, int, char*, int, int, int, int, int) $\rightarrow$ int

stampa_report (PQueue) $\rightarrow$ void

Specifica semantica

Tipo di riferimento: PQueue è il tipo che rappresenta una collezione organizzata di elementi di tipo Segnalazione gestiti tramite un Binary Heap, e monitorati attraverso una tabella Hash e tre array di liste per stato, urgenza e categoria.

Operatori:

newPQ () → q

-Post: $q = \{ \}$ (coda vuota con sotto-strutture allocate).

-Side Effect: Allocazione dinamica della memoria per la struttura principale c_PQ , per la tabella hash associata e inizializzazione a NULL dei puntatori degli indici.

emptyPQ (q) → r

-Post: $r = 1$ se $q = \{ \}$, $r = 0$ altrimenti.

DestroyPQueue (q) → void

-Post: $q = \text{NULL}$.

-Side Effect: Deallocazione permanente di tutta la memoria occupata dalla coda, dalla tabella hash e dalle liste dei tre indici secondari.

insert (q, key) → r

-Post: $r = 1$ se l'inserimento o l'aggiornamento ha avuto successo, $r = 0$ altrimenti.

-Side Effect: Se key non esiste nel sistema, viene inserita nell'array vet incrementando numel. Richiama le funzioni di riordinamento dell'heap sali o scendi.

delete (q, cod_id) → r

-Post: $r = 1$ se la rimozione ha avuto successo, $r = 0$ altrimenti.

-Side Effect: Se cod_id è presente e ha il flag rimuovibile impostato a 1, la segnalazione viene eliminata dall'array dell'heap decrementando numel. Il rispettivo ID viene rimosso dalle liste dei tre indici e la chiave viene cancellata dalla tabella hash. Richiama la funzione di riordinamento dell'heap scendi.

visualizzaTutte (q) → void

-Post: stampa a schermo tutte le segnalazioni presenti nell'heap.

visualizzaPerStato (q, stato_cercato) → void

-Pre: $1 \leq \text{stato_cercato} \leq 3$.

-Post: scorre la lista dell'indice di stato corrispondente e stampa le segnalazioni corrispondenti.

visualizzaPerUrgenza (q, urgenza_cercata) → void

-Pre: $1 \leq \text{urgenza_cercata} \leq 5$.

-Post: scorre la lista dell'indice di urgenza corrispondente e stampa le segnalazioni corrispondenti.

visualizzaPiuUrgenti (q, i) → void

-Pre: $i \leq$ numero di elementi in q.

-Post: Stampa i primi “i” nodi più urgenti.

visualizzaPerCodice (q, codice_cercato) → void

-Pre: $\text{codice_cercato} > 0$.

-Post: usa la tabella hash per trovare la posizione nell'heap della chiave e stampa i dati della segnalazione trovata.

visualizzaPerCategoria (q, categoria_cercata) → void

-Pre: $1 \leq \text{categoria_cercata} \leq 5$.

-Post: scorre la lista dell'indice di categoria corrispondente e stampa le segnalazioni correlate.

rendi_rimovibile (q, cod_id) → r

-Pre: cod_id presente nella hash table.

-Post: $r = 1$ se la modifica ha successo, $r = 0$ altrimenti.

-Side Effect: Ricerca la segnalazione tramite l'hash e modifica permanentemente il campo rimuovibile impostandolo a 1

cambia_stato (q, cod_id, nuovo_stato) → r

-Pre: cod_id presente nell'hashtable, $1 \leq \text{nuovo_stato} \leq 3$.

-Post: $r = 1$ se la modifica ha successo, $r = 0$ altrimenti.

-Side Effect: Modifica il campo stato della segnalazione trovata. Rimuove l'ID dal vecchio indice di stato e lo inserisce in testa nella lista del nuovo stato.

caricaDaFile (q, nomefile) → r

-Pre: nomefile valido.

-Post: legge le segnalazioni dal file di testo, le valida e le inserisce nella coda riordinando le strutture. $r = 1$ in caso di successo, $r = 0$ altrimenti.

-Side Effect: Modifica la memoria interna della coda inserendo elementi in heap, hash e liste; effettua operazioni di lettura sul file system.

insertSegnalazione (q, cod_id, nome, cogn, cat, desc, giorno, mese, anno, urg, stato) → r

-Post: $r = 1$ se la creazione e l'inserimento vanno a buon fine, $r = 0$ altrimenti.

-Side Effect: Valida i parametri, genera l'istanza di Segnalazione e la inserisce nella coda richiamando l'operatore insert.

stampa_report (q) → void

-Post: genera a schermo un riepilogo testuale complessivo sullo stato di occupazione e sui dati della coda.

4.4 ADT hashtable

Specifica sintattica

Tipi di riferimento: hashtable

Tipi usati: int, void

Operatori: newHashtable (int) → hashtable

hashFun (int, int) → int

hash_insert (hashtable, int, int) → int

hashSearch (hashtable, int) → int

hashDelete (hashtable, int) → int

DestroyHashtable (hashtable) → void

Specifica semantica

Tipo di riferimento: hashtable è il tipo che rappresenta una struttura dati per l'accesso e l'indicizzazione rapida delle informazioni, composta internamente da un valore di dimensione e un array di puntatori a strutture item collegate in caso di collisione.

Operatori:

newHashtable (size) → h

-Post: h rappresenta la tabella hash inizializzata con dimensione pari a size e l'array di puntatori azzerato. Se l'allocazione fallisce, h = NULL.

-Side Effect: Allocazione dinamica della memoria per la struttura della tabella e azzeramento dei puntatori dell'array interno.

hashFun (key, size) → indx

-Post: indx è il valore intero risultante dall'operazione di modulo (key % size), utile per mappare la chiave all'interno dei limiti dell'array della tabella.

hash_insert (h, key, nuova_pos_heap) → r

-Pre: array interno non nullo.

-Post: r = 1 se l'inserimento o l'aggiornamento è andato a buon fine, altrimenti r = 0

-Side Effect: Se la chiave key esiste già all'interno della tabella, ne aggiorna la posizione dell'heap con il valore nuova_pos_heap. Se non esiste, alloca dinamicamente un nuovo item associato e lo inserisce in testa alla lista dell'indice calcolato.

hashSearch (h, key) → pos

-Pre: array interno non nullo.

-Post: effettua la scansione della lista collegata all'indice associato alla chiave. Se la chiave key viene trovata, pos assume il valore numerico della sua posizione corrente nell'heap. Se non viene trovata, pos = -1.

hashDelete (h, key) → r

-Pre: array interno non nullo.

-Post: r = 1 se l'eliminazione ha avuto successo, r = 0 altrimenti.

-Side Effect: Se presente, scollega il nodo dalla lista concatenata associata, riorganizza i puntatori adiacenti e dealloca la memoria del nodo rimosso.

DestroyHashtable (h) → void

-Post: la tabella h non è più accessibile nel sistema.

-Side Effect: Scorre iterativamente l'intero array eliminando e liberando la memoria di ogni singola lista di item presente per poi deallocare permanentemente la struttura della tabella h.

4.5 File Gestione_Main

Specifica sintattica

Tipi usati: PQueue, int, FILE*, void

Operatori: void menu ()

void inserisci_file_main (PQueue)

void inserisci_main (PQueue)

int elimina_main (PQueue)

void cerca_stato_main (PQueue)

void cerca_urgenza_main (PQueue)

void vedi_prime_main (PQueue)

void cerca_id_main (PQueue)

void cerca_categoria_main (PQueue)

void rendi_rimovibile_main (PQueue)

int cambia_stato_main (PQueue)

void test_1 (FILE*)

void test_2 (FILE*)

void test_3 (FILE*)

void test_4 (FILE*)

void test_5 (FILE*)

void test_6 (FILE*)

void test_7 (FILE*)

void test_8 (FILE*)

Specifica semantica

Operatori:

menu () → void

-Post: Stampa a schermo il menu testuale interattivo contenente l'elenco delle opzioni disponibili per l'utente.

inserisci_file_main (q) → void

-Post: Richiede l'input del nome del file di testo e invoca l'operatore dell'ADT caricaDaFile per l'inserimento massivo delle segnalazioni.

-Side Effect: Pulisce il buffer di input corrente tramite la rimozione dei caratteri residui.

inserisci_main (q) → void

-Post: Legge da tastiera gli attributi richiesti per una nuova segnalazione, ne effettua la convalida preliminare dei limiti numerici e invoca l'operatore insertSegnalazione sulla coda q.

-Side Effect: Svuota ripetutamente il buffer di input a ogni lettura per prevenire errori di scansione delle stringhe.

elimina_main (q) → r

-Post: Acquisisce il codice ID e invoca l'operatore delete sulla coda q. Restituisce r = 1 in caso di errore critico, r = 0 altrimenti.

-Side Effect: Se l'operatore rileva un errore strutturale grave (valore di ritorno pari a 2), invoca direttamente la funzione di deallocazione DestroyPQueue liberando l'intera memoria.

cerca_stato_main (q) → void

-Post: Acquisisce il filtro numerico dello stato richiesto (1-3) e richiama l'operatore visualizzaPerStato per mostrare i record filtrati.

cerca_urgenza_main (q) → void

-Post: Acquisisce il filtro numerico del livello di urgenza richiesto (1-5) e richiama l'operatore visualizzaPerUrgenza per mostrare i record filtrati.

vedi_prime_main (q) → void

-Post: Acquisisce il numero intero di elementi desiderati e interroga la coda mediante l'operatore `visualizzaPiuUrgenti` per mostrare a schermo i primi nodi più urgenti.

`cerca_id_main (q) → void`

-Post: Legge il codice identificativo da tastiera e invoca l'operatore `visualizzaPerCodice` basato sull'interrogazione dell'hash table.

`cerca_categoria_main (q) → void`

-Post: Mostra il sotto-menu delle categorie disponibili, acquisisce la scelta dell'utente (1-5) e invoca l'operatore `visualizzaPerCategoria`.

`rendi_rimovibile_main (q) → void`

-Post: Acquisisce il codice ID della segnalazione desiderata e invoca l'operatore `rendi_rimovibile` per modificarne lo stato di protezione.

`cambia_stato_main (q) → r`

-Post: Legge l'ID della segnalazione e il valore del nuovo stato (1-3), convalidando l'input. Invoca l'operatore `cambia_stato` sulla coda `q`. Restituisce il codice di successo o fallimento dell'operazione.

`test_1 (f_results) → void`

-Pre: `f_results` puntatore a file valido e aperto in scrittura.

-Post: Esegue la prima suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.

-Side Effect: genera il file di output scrivendo al suo interno.

`test_2 (f_results) → void`

-Pre: `f_results` puntatore a file valido e aperto in scrittura.

-Post: Esegue la seconda suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.

-Side Effect: genera il file di output scrivendo al suo interno.

`test_3 (f_results) → void`

-Pre: `f_results` puntatore a file valido e aperto in scrittura.

- Post: Esegue la terza suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

test_4 (f_results) → void

- Pre: `f_results` puntatore a file valido e aperto in scrittura.
- Post: Esegue la quarta suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

test_5 (f_results) → void

- Pre: `f_results` puntatore a file valido e aperto in scrittura.
- Post: Esegue la quinta suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

test_6 (f_results) → void

- Pre: `f_results` puntatore a file valido e aperto in scrittura.
- Post: Esegue la sesta suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

test_7 (f_results) → void

- Pre: `f_results` puntatore a file valido e aperto in scrittura.
- Post: Esegue la settima suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

test_8 (f_results) → void

- Pre: `f_results` puntatore a file valido e aperto in scrittura.
- Post: Esegue l'ottava suite di test automatizzati sulle funzionalità del sistema e scrive l'esito della verifica all'interno del file `f_results`.
- Side Effect: genera il file di output scrivendo al suo interno.

5 Razionale dei Casi di Test

I test svolti sono 8, ognuno riguardante una categoria di azioni specifica. Queste, in ordine, sono:

1. Dati errati e corretto inserimento
2. Ricerca per id
3. Cambiamento di stato
4. Priorità
5. Ricerca filtrata
6. Report
7. Eliminazione
8. Visione delle più urgenti

5.1 Dati errati e corretto inserimento

All'interno del primo Test case sono presenti 3 segnalazioni valide e 6 segnalazioni non valide, ognuna per un motivo diverso.

Questi sono: codice id, categoria, stato, urgenza e data non validi, codice id duplicato.

Inoltre è presente una segnalazione con generazione automatica dell'id.

5.2 Ricerca per id

Nel secondo Test case si hanno semplicemente 2 segnalazioni valide, con 2 ricerche fallimentari e 2 ricerche con successo.

5.3 Cambiamento di stato

Dentro il terzo Test case ci sono 4 segnalazioni, di cui due con stato 1, una con stato 2 e una con 3. Gli stati cambieranno da 1 a 2, da 2 a 3, da 3 a 1 e da 1 a 1.

5.4 Priorità

Nel quarto Test case sono presenti 6 segnalazioni tutte valide con urgenze e date diverse. Esso si occupa di vedere che esse siano correttamente stampate in ordine di priorità.

5.5 Ricerca filtrata

All'interno del quinto Test case sono presenti 15 segnalazioni, tutte valide, ognuna con un possibile dato con cui far avvenire una ricerca filtrata diversa. Il test verifica che siano tutte correttamente ricercate

5.6 Report

Il sesto Test case presenta 12 segnalazioni, ancora una volta tutte valide, di diverso tipo. Il test verifica che il report sia coerente con i dati forniti.

5.7 Eliminazione

Nel settimo Test case sono presenti solo 2 segnalazioni, valide, di cui una riceverà prima un tentativo di eliminazione con flag a 0, poi il flag verrà modificato a 1 e sarà poi finalmente eliminata. Il test verifica che la segnalazioni si comporti come previsto.

5.8 Visione delle più urgenti

L'ottavo e ultimo Test case contiene 12 segnalazioni valide, e verifica che richiedendone la stampa di un numero specificato, queste vengano correttamente fornite. Il test verifica che con input validi e non, il programma abbia il comportamento previsto.